Examen de Metodología de la Programación
Curso 2025-2026, Convocatoria I (Enero)

Universidad de Huelva

Apellidos:_____Nombre:_____

Sección Informativa: **Duración del examen: 2 horas**

C++. Dada la clase Fecha, la clase Documento y su clase hija Libro:

```
class Fecha {
  int dia;
  char *mes;
  int anio;
public:
  ...

};
```

```
class Documento {
  static int contador;
  int idDocumento;
  char *titulo;
public:
  ...

};
```

```
class Libro: public Documento {
  Fecha fechaEdicion;
  char *editorial;
  double precio1, precio2;
public:
  ...
};
```

```
int main() {
  const Fecha f1(23,"abril",2026); //Fecha f4; //no permitido
  cout << f1.getDia() << " de " << f1.getMes() << " de " << f1.getAnio();
  cout << " Faltan " << (int)f1 << " días para terminar el mes de " << f1.getMes() << endl;
  Fecha f2(0,"junio",2001), f3(31,"febrero",2005);
  Fecha *f=new Fecha(5,"marzo",2020);
  if (f2!=f3)
    *f=f3;
  cout << f1 <<", "<< f2 <<","<< f3 <<","<< *f << endl;
  delete f;
  Documento d1("La Odisea"), d2(d1);
  Documento *d= new Documento("Cuentos");
  cout << d1 << endl << d2 << endl << *d << endl;
  if (d1==d2) cout << "d1 y d2 son identicos" << endl;
  else cout << "d1 y d2 no son identicos" << endl;
  d2=d1; //crea un clon identico
  cout << d1 << endl << d2 << endl << *d << endl;
  if (d1==d2)
    cout << "d1 y d2 son exactamente iguales" << endl;
  delete d;
  cout << "--LIBROS--" << endl;
  Libro i1("Vida", f1, "Planeta", 10.25, 13.00);
  const Libro i2("Dracula", f2, "Alfaguara", 20.50);
  Libro i3("Sin Person", f2, "Planeta", 10.00, 15.00);
  cout << i1 << endl << i2 endl << i3 << "\n---\n";
  i1=i2; //crea un clon identico
  Libro::subirPrecio(i1, 1.00);
  Libro::subirPrecio(i3, 0.50);
  Libro::informe(i1); Libro::informe(i2); Libro::informe(i3);
  if (i1!=i2)
    cout << "i1 y i2 no son exactamente iguales" << endl;
  return 0;
}
```

```
23 de abril de 2026
faltan 7 dias para terminar el mes de abril
23/abril/2026, 1/junio/2001, 28/febrero/2005, 28/febrero/2005
1: La Odisea
2: La Odisea
3: Cuentos
d1 y d2 no son identicos
1: La Odisea
1: La Odisea
3: Cuentos
d1 y d2 son exactamente iguales
--LIBROS--
4: Vida - 23/abril/2026 Ed. Planeta, 10.25€
5: Dracula - 1/junio/2001 Ed. Alfaguara, 20,50€
6: Sin Perdon - 1/junio/2001 Ed. Planeta, 10,00€
---
5: Dracula - 1/junio/2001 Ed. Alfaguara, 21,50€  ( precio unico )
5: Dracula - 1/junio/2001 Ed. Alfaguara, 20,50€  ( precio unico )
6: Sin Perdon - 1/junio/2001 Ed. Planeta, 10,50€ ( precio2: 15.50 )
I1 y i2 no son exactamente iguales
```

1. Implementa la clase Fecha y los métodos que **estrictamente** están en el main( ). **(2.25 puntos)**

   Nota: Para simplificar el código, suponed que todos los meses tienen 30 días, excepto "febrero" que tendrá 28 (no existen años bisiestos). Además, suponed que el mes y año siempre se introduce de forma correcta, pero el día no. Por tanto, el constructor debe comprobar que el día es correcto (si no lo es, se debe poner como día el más cercano al indicado).

   Nota: **Como no se usa estrictamente en el main() NO implementamos el constructor de copia**

2. Implementa la clase Libro **usando el menor número de constructores posibles** y los métodos **estrictamente necesarios** para que el main() pueda ejecutarse. **Los métodos** (set, constructor copia) **que estrictamente no se invocan** no hay que implementarlos y **no se pueden usar**. **(3.50 puntos)**

   Nota: Suponed que los métodos de las clases Fecha y Documento que **estrictamente se usan** en el main() están implementados y funcionan correctamente. **No es posible añadir ningún método adicional a dichas clases (por tanto, la clase Fecha no tiene implementado el constructor de copia).**

   Un libro puede tener 2 precios distintos. Por pantalla se debe mostrar únicamente el primer precio, excepto cuando se invoca el método `informe` que debe mostrar los 2 precios que tenga o indicar que el precio es único.

SOLUCION: clase Fecha **(2.25 puntos)**

La clase Fecha **NO tiene un constructor de copia** ya que en main() no se usan estrictamente: **el que genera de oficio el compilador no funciona correctamente, por lo que debemos evitar pasar Fechas por copia (las pasaremos por referencia siempre)**.

```
class Fecha {
  int dia;
  char *mes;          //los atributos punteros hay que reservar memoria con new y liberar con delete
  int anio;           //las cadenas de caracteres se copian con strcpy y se comparan con strcmp
public:
  Fecha(int d, const char *m, int a) {                                                    //0.60
    int tope=30;                  //se supone que todos los meses tienen 30 dias
    if (strcmp(m, "febrero")==0)
      tope=28;                    //menos febrero que son 28
    if (d<1) d=1;
    else if (d>tope) d=tope;
    dia=d;
    mes=new char[strlen(m)+1];
    strcpy(mes, m);
    anio=a;
  }

  ~ Fecha () { delete [] mes; }                                                           //0.10

  int getDia() const { return dia; }                                                      //0.05
  const char* getMes() const { return this->mes; } //cons char * para que sea ROBUSTA     //0.10
  int getAnio() const { return anio; }                                                    //0.05

  bool operator!=(const Fecha &f) const {                                                 //0.30
    return (dia!=f.dia || strcmp(mes,f.mes)!=0 || anio!=f.anio);
  }
//otra forma de implementar el operator !=
  bool operator!=(const Fecha &f) const {
    if (dia==f.dia && strcmp(mes,f.mes)==0 && anio==f.anio)
      return false;
    else
      return true;
  }


  Fecha& operator=(const Fecha& f) {                                                      //0.50
    if (this != &f) {     //lo primero que hay que ver es si el operador izq y derecho son diferentes
      dia=f.dia;
      anio=f.anio;
      delete [] mes;
      mes=new char[strlen(f.mes)+1];
      strcpy(mes, f.mes);
    }
    return *this;
  }

//el operador de casting (int) convierte una Fecha a un int
  operator int() const {                                                                  //0.30
    int tope=30;              //se supone que todos los meses tienen 30 dias menos febrero que son 28
    if (strcmp(mes, "febrero")==0)
      tope=28;
    return tope-dia;          //devuelve un entero que indica cuantos dias quedan para terminaer el mes
  }


  friend ostream& operator<<(ostream &s, const Fecha &f) {                                //0.25
    s << f.dia << "/" << f.mes << "/" << f.anio;
    return s;
  }
};
```

SOLUCION: clase Libro **(3.50 puntos)**

Para implementar la clase libro tenemos que tener claro qué métodos tienen las clases Fecha y Documento ya que en el enunciado se dice "suponed que los métodos de las clases Fecha y Documento que **estrictamente se usan** en el main() están implementados y funcionan correctamente. **No es posible añadir ningún método adicional a dichas clases (por tanto, la clase Fecha no tiene implementado el constructor de copia)".**

Viendo el main() deducimos que los métodos de la clase Fecha y de la clase Documento son **estrictamente** los siguientes:

```
class Fecha {
  int dia, anio;
  char *mes;
public:
  Fecha(int d, const char *m, int a);
  Fecha(const Fecha &f);
  ~Fecha();
  Fecha& operator=(const Fecha& f);
  int getDia() const
  const char* getMes() const;
  int getAnio() const
  operator int() const;
  bool operator!=(const Fecha &f) const;
  friend ostream& operator<<(ostream &s,
                             const Fecha &f);
};
```

```
class Documento {
  static int contador;
  int idDocumento;
  char *titulo;
public:
  Documento(char *t);
  Documento(const Documento& d);
  ~Documento();
  Documento& operator=(const Documento& d);
  bool operator==(const Documento &d) const;
  friend ostream& operator<<(ostream &s,
                             const Documento &d);
};

int Documento::contador=1;
```

La clase Fecha NO tiene un **constructor de copia** y no lo podemos programar por lo que debemos evitar pasar Fechas por copia (las pasaremos por referencia siempre).
Para implementar la clase Libro "no es posible añadir ningún método adicional a las clases Fecha y Documento". Además, la clase Libro no puede usar el constructor de copia (los libros los pasaremos por referencia siempre).

```
class Libro: public Documento { //el constructor Libro debe llamar al constructor Documento (su padre)
                                //en la zona de inicializadores y pasarle al padre lo que es suyo
  Fecha fechaEdicion; //los atributos que son objetos hay que ponerlos en la zona de
                      //inicializadores del constructor (uso el constructor de copia de Fecha)
  char *editorial;    //los atributos punteros hay que reservar memoria con new y liberar con delete
  double precio1, precio2;
public:
//hay que hacer un único constructor con parámetros opcionales
  Libro (char *t, Fecha f, char *e, double p1, double p2=0): Documento(t),
    fechaEdicion(f.getDia(),f.getMes(),f.getAnio()) {
    editorial=new char[strlen(e)+1];
    strcpy(editorial, e);
    precio1=p1                                                                    //0.70
    precio2=p2;
  }
                                                                                  //0.10
  ~ Libro () { delete [] editorial; } //el destructor de Libro libera lo que es estrictamente suyo, de
                                      //liberar loque hereda del padre se ocupa el destructor Documento

//el operator= de Libro clona lo que es suyo y lo que es del padre (Documento) lo clona el padre
  Libro& operator=(const Libro& li) {                                             //0.70
    if (this != &li) { //lo primero que hay que ver es si el operador izq y derecho son diferentes
      Documento::operator=(li); //Libro debe llamar al operator= del padre para que clone su parte
      fechaEdicion=li.fechaEdicion; //uso el operator= de Fecha
      delete [] editorial;
      editorial=new char[strlen(li.editorial)+1];
      strcpy(editorial, li.editorial);
      precio1=li.precio1;
      precio2=li.precio2;
    }
    return *this;
  }

//informe muestra los precios adicionales del libro (y si no los tiene, indica que es precio único)
  static void informe(const Libro &li) {                                          //0.30
    cout << li;
    if (li.precio2==0)
      cout << " (precio unico)\n";
    else
      cout << " (precio2: " << li.precio2 << ")" << endl;
  }

  static void subirPrecio(Libro &li, double d) { //li hay que pasarlo por referencia para que los
    li.precio1+=d;                               //cambios afecten al original                //0.40
    if (li.precio2!=0)
      li.precio2+=d;
  }
```

```
//el operator!= de Libro compara lo que es suyo y lo que es del padre (Documento) lo compara el padre
  bool operator!=(const Libro &li) const {                                                    //0.70

    bool mismosPrecios=(precio1==li.precio1 && precio2==li.precio2);

    if (Documento::operator==(li) == false || fechaEdicion!=li.fechaEdicion ||
        strcmp(editorial, li.editorial)!=0 || mismosPrecios == false)
      return true;
    else
      return false;
  }

//otra forma de implementar el operator !=
  bool operator!=(const Libro &li) const {

    if (Documento::operator==(li)==true && (fechaEdicion!=li.fechaEdicion)==false &&
            strcmp(editorial, li.editorial)==0 && precio1==li.precio1 && precio2==li.precio2)
      return false;
    else
      retrun true;
  }



//el operator << de Libro solo debe mostrar el primero de los 2 posibles precios del libro   //0.60
//el operator << de Libro muestra lo que es suyo y lo que es del padre (Documento) lo muestra el padre
  friend ostream& operator<<(ostream &s, const Libro &li) {        //al castear li a Documento hago
    s << (Documento &)li << " - " << li.fechaEdicion << " Ed. "    //que el operator<< que se ejecute
      << li.editorial << ", " << li.precio1 << "€";                //sea el de la clase Documento, que
    return s;                                                      //se encarga de mostrar lo que es
  }                                                                //de Documento
};
```

3. **(JAVA)** Una empresa de protección de riesgos está desarrollando un programa para gestionar el sistema de alarmas de los campos universitarios. Dado que los edificios pueden presentar diferentes características térmicas según su construcción han decidido diseñar el siguiente código. Indica los posibles errores del siguiente código justificando el motivo. **(1.00 puntos)**

```
1.  public final class Edificio {
2.    private final int espacios;
3.    private boolean emergencia;
4.
5.    public Edificio(int espacios) {
6.      this.espacios = espacios;
7.      emergencia = false;
8.    }
9.    //El 30% de los espacios son despachos
10.   public static void getNDespachos() {
11.     return espacios * 0.3;
12.   }
13.   public final boolean setAlerta(boolean e) {
14.     this.emergencia = e;
15.   }
16.   public final void alertaIncendio() {
17.     if (emergencia)
18.       System.out.println("Evacuen edificio");
19.     else
20.       System.out.println("Edificio Seguro");
21.   }
22.   public abstract void direccion();
23. }
```

```
24. public class ETSI implements Edificio {
25.   public int temperatura;
26.
27.   public ETSI(int temperatura, int espacios){
28.    this.temperatura = this.temperatura;
29.    super(espacios);
30.   }
31.
32.   public final void alertaIncendio() {
33.     super.setAlerta(temperatura >= 50);
34.     if (emergencia){
35.       System.out.println("Evacuen ETSI");
36.     }else{
37.       System.out.println("ETSI Seguro");
38.     }
39.   }
40.
41.   public void direccion() {
42.     System.out.println("La direccion de la
43.     ETSI es: Avda. de las Fuerzas Armadas");
44.   }
45.
46. }
```

SOLUCION:

```
1.  public final abstract class Edificio {
2.    private final int espacios;
3.    private boolean emergencia;
4.
5.    public Edificio(int espacios) {
6.      this.espacios = espacios;
7.      emergencia = false;
8.    }
9.    //El 30% de los espacios son despachos
10.   public static int getNDespachos() {
11.     return espacios * 0.3;
12.   }
13.   public final boolean setAlerta(boolean e) {
14.     this.emergencia = e;
14.     return e;
15.   }
16.   public final void alertaIncendio() {
17.     if (emergencia)
18.       System.out.println("Evacuen edificio");
19.     else
20.       System.out.println("Edificio Seguro");
21.   }
22.   public abstract void direccion();
22.   public boolean getEmergencia() {
22.     return emergencia;
22.   }
23. }
```

```
24. public class ETSI extends Edificio {
25.   private int temperatura;
26.
27.   public ETSI(int temperatura, int espacios){
28.     super(espacios);
29.     this.temperatura = temperatura;
30.   }
31.
32.   public final void alertaIncendio() {
33.     super.setAlerta(temperatura >= 50);
34.     if (getEmergencia(){
35.       System.out.println("Evacuen ETSI");
36.     }else{
37.       System.out.println("ETSI Seguro");
38.     }
39.   }
40.
41.   public void direccion() {
42.     System.out.println("La direccion de la
43.     ETSI es: Avda. de las Fuerzas Armadas");
44.   }
45.
46. }
```

La empresa anterior desarrolla también un software gráfico para los planos de los edificios. Dado el siguiente código, responde a las cuestiones planteadas:

```
public class Punto2D {
  private int x;
  private int y;

  public Punto2D(int x, int y) {
    this.x = x;
    this.y = y;
  }

  public int getX() { return x; }
  public int getY() { return y; }

  public void setX(int x) {
    this.x = x;
  }

  public void setY(int y) {
    this.y = y;
  }

  @Override
  public String toString() {
    return "(" + x + ", " + y + ")";
  }
}
```

```
public class Figura {
  private String nombre;
  private Punto2D ubicacion;

  public Figura(String nombre, Punto2D ubicacion) {
    this.nombre = nombre;
    this.ubicacion = ubicacion;
  }

  public String getNombre() { return nombre; }
  public Punto2D getUbicacion() { return ubicacion; }

  public void setUbicacion(Punto2D nuevaUbicacion) {
    this.ubicacion = nuevaUbicacion;
  }

  public boolean plano_terminado() { return true; }
}

public class PruebaFigura {
  public static void main(String[] args) {
    Figura pixel = new Figura("pixel", new Punto2D(5, 10));
    System.out.println(pixel.getNombre() + " está en " +
                       pixel.getUbicacion());
  }
}
```

4. Implementa el método clone en la clase Figura. **(0.75 puntos)**

Implementación de clone() en la clase Figura

```
public Figura(Figura f) {
    nombre = f.nombre;
    ubicacion = new Punto2D(f.ubicacion);

}

public Object clone() {
  return new Figura(this);
}
```

Implementación de clone() en la clase Figura (objeto compuesto):

```
public Object clone() {
  Figura obj=null;
  try{
    obj=(Figura)super.clone();
    obj.ubicacion=(Punto2D)obj.ubicacion.clone();
  }catch (CloneNotSupportedException ex) {
    System.out.println(" no se puede duplicar");
  }
  return obj;
}
```

Implementación constructor en la clase Punto2D

```
public Punto2D (Punto2D p) {
    this.x = p.x;
    this.y = p.y;
}
```

Implementación de clone() en la clase Punto2D (objeto simple):

```
public Object clone() throws CloneNotSupportedException {
  Object obj=null;
  try{
    obj=super.clone();
  }catch (CloneNotSupportedException ex) {
    System.out.println(" no se puede duplicar");
  }
  return obj;
}
```

Tanto la solución de la columna izquierda como la de la derecha son válidas

5. Sin modificar nada del código, indica que atributos de la clase **Punto2D** y **Figura** pueden ser **final** de modo que el comportamiento de las clases siga siendo el mismo. Justifica tu respuesta. **(0.25 puntos)**

Solo puede ser **final** el atributo nombre de la clase Figura

6. ¿Es posible modificar en el main() la ubicación de **pixel** (ejemplo, a la coordenada x = 1, y = 4) sin usar el método **setUbicacion( )**? Justifica tu respuesta. **(0.50 puntos)**

```
public static void main(String[] args) {
  Figura pixel = new Figura("pixel", new Punto2D(5, 10));
  Punto2D ubi = pixel.getUbicacion();
  ubi.setX(1);  ubi.setY(4);
  System.out.println("pixel está en " + pixel.getUbicacion());
}
```

7. Si el atributo **ubicacion** de la clase Figura fuera **final**, ¿el método **setUbicacion( )** funcionaría o habría que modificarlo? Razona tu respuesta. **(0.50 puntos)**.

```
public void setUbicacion(Punto2D nuevaUbicacion) {
  this.ubicacion.setX( nuevaUbicacion.getX() );
  this.ubicacion.setY( nuevaUbicacion.getY() );
}
```

8. Modifica los **setX** y **setY** para que devuelva una excepción en caso de introducir una coordenada negativa. Haz un main( ) de ejemplo que capture la excepción. **(0.50 puntos)**

```
public void setX(int x) throws Exception {   //idem setY
  if (x < 0)
    throw new Exception();
  this.x = x;
}

public static void main(String[] args) {
  try {
    Punto p = new Punto2D(5, 10));
    p.setX(-5); p.setY(-8);
  } catch (Exception ex) {
  System.out.println("coordenada negativa");
}
```

9. Sin cambiar nada de la clase **Punto2D**, modifica el código de la clase **Figura** para que sea seguro, es decir, que los objetos **Figura** que se creen solo puedan cambiar el valor de sus atributos a través de los métodos sets correspondientes. **(0.75 puntos)**

```
public class Figura {
  private String nombre;
  private Punto2D ubicacion;

  public Figura(String nombre, Punto2D ubicacion) {
    this.nombre = nombre;
    this.ubicacion = new Punto2D( ubicacion.getX(), ubicación.getY() );
  }

  public String getNombre() { return nombre; }
  public Punto2D getUbicacion() { return new Punto2D( ubicacion.getX(), ubicacion.getY() ); }

  public void setUbicacion(Punto2D nuevaUbicacion) {
    this.ubicacion = new Punto2D( nuevaUbicacion.getX(), nuevaUbicación.getY() );
  }

  public boolean plano_terminado() {
    return true;
  }
}
```

Como la clase Punto2D es mutable, por seguridad nunca debemos devolver la ubicación original sino una copia. De igual forma la ubicación debe apuntar a una copia de la ubicación que nos pasen por parámetro.